

Meeting Intelligence Agent

Task	Meeting Intelligence Agent
Format	Working prototype with live deployment
Submission	Git Repository + Live URL + Screen Recording

Problem Statement

Sales teams, account managers, and agencies walk into external calls underprepared. They spend 20-30 minutes before every meeting searching for the company, scanning for recent news, and piecing together talking points — only to still miss something important by the time the call starts.

Build an autonomous agent that monitors a user's calendar, identifies upcoming external calls, researches the companies involved, and presents a ready-to-use intelligence brief for each meeting — without the user having to ask.

The user should open a dashboard, see all their calls for the day or week, and find a fully researched brief for each one — prepared and waiting before the meeting begins.

What to Build

The solution has three connected components that must work together end-to-end.

1. Calendar Listener

Connect to the user's calendar via an external tool integration. Fetch upcoming meetings for the current day or week. For each meeting, extract the company name or domain from the meeting title, description, or attendee email addresses.

Handle ambiguous or incomplete meeting data — if a meeting is titled 'Sync with Sarah' and Sarah's email is sarah@acme.com, the agent should infer the company as Acme and proceed.

2. Research Pipeline

For each identified company, run an autonomous research process using external tool integrations. Fetch live data from at least two external sources and synthesize into a structured brief covering:

- What the company does — in plain language, not marketing copy

- Recent news, product launches, funding, or announcements from the last 60-90 days
- Tech signals — stack, tools, infrastructure, or open-source activity where detectable
- Inferred pain points based on their stage and public activity

3. Dashboard

A simple web dashboard displaying all upcoming calls, each as a card with the research brief. Each card must show:

- Meeting title, time, and attendees
- Company brief, recent activity, and tech signals
- 2-3 suggested talking points

The dashboard must be live at a public URL and update automatically as new meetings are detected.

Example Scenarios

Scenario A — Clear Company Name

```
Calendar event: "Demo call with Linear"
Attendees:      john@linear.app
Agent infers:   Company = Linear (linear.app)

Dashboard card:
  Company:      B2B project management tool for engineering teams.
                Known for speed, minimalist UX, and keyboard-first design.
  Recent News:  Launched 'Linear Insights' — AI-powered sprint analytics (Apr
2025).
                Raised Series B at $400M valuation (Feb 2025).
  Tech Signals: Heavy TypeScript. Active open-source. Likely on AWS + Vercel.
  Pain Points:  Scaling coordination beyond 50 engineers.
  Talking Points:
    - How are they thinking about AI within their own product workflow?
    - What does their eng-to-PM ratio look like today?
    - Have they explored tooling for sprint retrospectives?
```

Scenario B — Company Inferred from Attendee Email

```
Calendar event: "Intro call with Priya"
Attendees:      priya@growthsignal.io
Agent infers:   Company = GrowthSignal (growthsignal.io)

Dashboard card:
  Company:      Early-stage B2B SaaS in revenue intelligence. Founded 2023.
```

Recent News: Product update on blog (Mar 2025). No funding found. Pre-seed.
Tech Signals: React frontend. Uses Segment and HubSpot per job listings.
Pain Points: ICP definition, first enterprise customers, proving retention.
Talking Points:
- What does their current customer acquisition motion look like?
- Are they seeing repeatability in their sales cycle yet?
- What does success look like in the next 6 months?

Scenario C — Ambiguous or Missing Data

Calendar event: "Catchup - Ravi"
Attendees: ravi@gmail.com

Agent response: Cannot infer company — personal email, no company in title.
Dashboard card: Meeting shown with time and attendees.
Brief shows: 'Company could not be identified for this meeting.'
No crash. No blank card. Graceful fallback.

Deliverables

All of the following are required for a complete submission:

Deliverable	What It Must Include
Git Repository	Full source code in a public GitHub repo. README with setup instructions, environment variables, and how to run locally.
Live URL	A deployed, publicly accessible dashboard URL. Must be live and functional at submission.
Screen Recording	3-5 minute recording demonstrating the complete end-to-end flow. Details below.
Submission Note	3-5 sentences on what you would improve or extend given more time.

Screen Recording Must Show

1. Dashboard open with list of upcoming meetings
2. Agent detecting a calendar event and identifying the company
3. Research pipeline running and populating the meeting card
4. Final dashboard card with all research content visible

Constraints

- Agent must fetch live data — no static or hardcoded research
- Calendar and research must use external tool integrations — not manual input
- At least two external data sources must be used in the research pipeline
- Dashboard must update automatically when new meetings are detected
- Tool failures must be handled gracefully — no crashes, no blank cards
- Live URL must be accessible at the time of review
- When company cannot be identified, show a clear fallback state — not an error or empty card
- Research output must be synthesized and structured — not a raw dump of search results
- The agent must decide what to look up based on available signals — not follow a hardcoded sequence

Tool usage Guidelines

- You are free to use any AI coding IDEs (such as Cursor, Windsurf, or VS Code with custom extensions) and development tools required to complete the assignment.
- Your implementation and agent architecture are not restricted to any specific Model Context Protocols (MCPs) or standard patterns.
- Leverage any framework, API library, or custom workflow system that successfully achieves the core requirements.

Submission

Share the following via email or the submission form:

- GitHub repository link
- Live dashboard URL
- Screen recording link (Loom, Google Drive, or any accessible platform)
- A brief note covering: your architecture decisions, what you would improve with more time, and any trade-offs you made

Submission deadline: This assignment is expected to require approximately **4 hours of focused effort**. You may submit your solution within **2 days from the date this task is shared with you**.

What Happens After Submission

Once you share your implementation, we will schedule a review call where you will walk us through the architecture, key decisions, and provide a live demo of the solution. During the discussion, we will introduce additional scenarios, edge cases, or small modification requests to evaluate your problem-solving approach, technical depth, and adaptability. This helps us understand how you think through real-world challenges beyond the initial implementation.

Good luck—we're looking forward to your submission!